AI Logs – Sprint 2

1. Planning US-06 and US-07 on the existing codebase

Purpose of AI Use: Understanding the code my teammates had pushed (including the new CI pipeline) and planning how to build Job Application Submission and Tracking on top of it.

Chat Link or Prompt/Response:

Prompt: here are the instructions for sprint 2. I'm responsible for the 3rd point: Job Application Submission and Tracking. I attached the user stories and issues that are related. I'm sharing with 2 people. My teammates already started sprint 2 and they added a CI pipeline. I'll show you but first we need to pull the new modifications help me do that and I'll send you the actual code [attached Sprint 2 instructions and a screenshot of the US-06/US-07 issues]

Then: here is what we got for now [attached zip of the repository]

Response: I've gone through the code. Your teammates set things up so each feature mostly stays in its own files, which makes your part easier. Here's what matters for US-06 and US-07.

What you can build on

- **Login:** `get_current_user` in `routers/auth.py` protects routes, and `current_user.role` tells you whether the user is a `job_seeker` or a `recruiter`. You need this for Task 7.1, where only recruiters can change a status.
- **Primary resume:** The resume feature already keeps one primary resume per user, explicitly so applications can use it by default. That's most of the "resume selection" in Task 6.3.
- **One file per feature:** You'll add `models/application.py`, `schemas/application.py` and `routers/application.py`, then one import line in each `__init__.py` and one `include_router` in `app.py`.
- **Tests:** Create `tests/test_applications.py` and use the existing `client` and `make_user(role="recruiter")` helpers. CI picks the file up automatically.

The blocker: there's no jobs table yet. An application links a job seeker to a job posting, but nobody has created a `Job` model so far. It belongs to whoever owns feature 2 (job search) or feature 4 (recruiter postings). [...] If nobody has started it, offer to write a minimal `Job` model yourself (id, recruiter_id, title, company, location, description, created_at) and let the jobs owner add to it.

My plan for 6.1 once that's settled: an `applications` table with `id`, `job_id` → jobs, `applicant_id` → users, `resume_id` → resumes, `status` (one of Applied, Under Review, Interview, Offered, Rejected; starts as Applied), `applied_at` and `updated_at`, and a unique constraint on (job_id, applicant_id), so the database itself blocks duplicate applications (Task 6.2). For Task 7.3, an `application_status_history` table: application_id, old_status, new_status, changed_by, changed_at.

One gotcha from their code: `create_all()` never changes tables that already exist. New tables like yours are fine, but changing an existing table on Supabase means running an `ALTER TABLE` by hand, the way they did for `is_primary`.

AI-Suggested Content:

- Git commands to pull the CI changes on main before branching.
- A list of existing components to reuse (authentication, primary resume, test fixtures).
- A schema plan for the applications and status history tables.
- Identification of the missing Job model as a blocker.

Validation: Compared the analysis with the repository's folders and the README, and confirmed that no job model existed yet. The first pull instructions used the wrong folder (backend/ instead of Sprint 2/backend), which I reported and the AI corrected.

Decision: Partially accepted.

Reflection: The analysis saved me time getting into code I hadn't written, and it caught the missing job model before I started. I didn't follow the advice to wait for the teammate who would own the job model, because that would have blocked my whole user story.

Responsible Person: Diego Chidiac

2. Task 6.1: Design database schema for applications

Purpose of AI Use: Creating the applications table (and a temporary jobs table) with the database rules the feature needs, plus automated tests.

Chat Link or Prompt/Response:

Prompt: i couldnt do this: Reinstall dependencies in case the CI work changed them. i dont want to wait for the job model let's do it tgt and move forward step by step. For my user story 6, my teammate is responsible for the frontend

Response: Your dependency step failed because the folders are `Sprint 2/backend` and `Sprint 2/frontend`, not `backend/` at the root. [...] First I'll write the Job and Application models and test them against a real Postgres here.

```
APPLICATION_STATUSES = ("Applied", "Under Review", "Interview", "Offered", "Rejected")
UniqueConstraint("job_id", "applicant_id", name="uq_application_job_applicant")
CheckConstraint("status IN ('Applied', ...)", name="ck_application_status")
job_id    = Column(Integer, ForeignKey("jobs.id", ondelete="CASCADE"), nullable=False)
resume_id = Column(Integer, ForeignKey("resumes.id", ondelete="SET NULL"), nullable=True)
```

The constraint works, but the pg8000 driver reports a CHECK violation as a different error type. I'll fix the test to accept it:

```
# pg8000 reports CHECK violations as ProgrammingError (not IntegrityError),
# so catch their common parent and check it was OUR constraint that fired.
with pytest.raises(DBAPIError, match="ck_application_status"):
```

Task 6.1 is done. I ran it against a real Postgres and all 67 tests pass: your teammates' 56 still pass, plus 11 new ones.

- **`models/job.py`**: a basic jobs table [...] The job-search and recruiter teammates can add columns to it later. Tell the group you created it so nobody makes a second one.
- **`models/application.py`**: the applications table.
  - **No duplicate applications:** the same person can't apply twice to the same job.
  - **Fixed statuses:** only Applied, Under Review, Interview, Offered or Rejected can be saved, and new applications start as Applied.
  - **Resume deletion:** if a job seeker deletes a resume they applied with, the application stays and just has no resume attached.
  - **Job deletion:** deleting a job deletes its applications too.
- **`tests/test_application_model.py`**: tests that check each of those rules.

[attached ai coded-files: models/job.py, models/application.py, models/__init__.py, tests/test_application_model.py]

AI-Suggested Content:

- An Application model with a unique (job, applicant) constraint and a status CHECK constraint.
- SET NULL on resume deletion and CASCADE on job deletion.
- A temporary minimal Job model.

- 11 database tests.
- Corrected setup commands for the Sprint 2 folder layout.

Validation: The AI ran the tests against a real PostgreSQL database. The first run failed: the pg8000 driver reports CHECK violations as ProgrammingError, not IntegrityError. The test was corrected to check the constraint name, and all 67 tests passed. I then read every file before committing (see entry 4).

Decision: Accepted, with the temporary job model kept in a separate commit.

Reflection: Building on a temporary job model let me move forward instead of waiting, but it created a risk of two different jobs tables, which I had to manage later (entries 8 and 9). The failing test showed why the AI's code needs to be run, not just read.

Responsible Person: Diego Chidiac

3. Task 6.2: Implement application submission API endpoint, preventing duplicates

Purpose of AI Use: Building the endpoint that lets a job seeker apply to a job, and the list of their own applications.

Chat Link or Prompt/Response:

Prompt: my branch is us06-application-submission, show me how I should implement the application submission API endpoint, preventing applications to the same posting.

Response: `POST /api/applications`: a job seeker applies to a job. Body: `{ "job_id": 5 }`, optionally with `"resume_id": 12`. It runs these checks in order. The first one that fails stops the request:

Check	If it fails
Logged in?	401
A job seeker (not a recruiter)?	403
Job exists?	404
Job still open?	400
Already applied to this job?	409 "You have already applied to this job."
Resume chosen? Then it must exist (404) and be theirs (403)	
No resume chosen? Then use their primary; if they have none	400 "Upload a resume before applying"

`GET /api/applications/mine`: the job seeker's own applications, newest first, each with job title, company, resume name and status. This covers the "track application status" part of feature 3 [...]

- **Duplicates are blocked twice.** The 409 check gives a clean message in the normal case. If two requests arrive at the exact same moment (a double-click), both can pass that check. Then the database rule from 6.1 rejects the second one, and the code turns that rejection into the same 409 instead of a crash.
- **The client can't set the status.** If someone sends `"status": "Offered"` in the body, the request is rejected. Only a recruiter will be able to change the status, through 7.1.

The whole file shows as changed: these files use Windows line endings, and my edit script converted them. That would make your PR look like you rewrote every line. Fixing it so only the real lines differ: [...] All 84 tests pass: the 67 from before plus 17 new ones.

[attached ai coded-files: routers/application.py, schemas/application.py, tests/test_applications.py, app.py, schemas/__init__.py]

AI-Suggested Content:

- POST /api/applications with ordered validation checks.
- GET /api/applications/mine.
- Two-layer duplicate prevention (application check + database constraint).
- 17 API tests.

Validation: 84 tests passed against a real PostgreSQL database. The AI noticed its own edits had converted Windows line endings and fixed them. Before committing I ran

git diff --stat to confirm that only the real lines had changed (2 in app.py, 4 in schemas/__init__.py).

Decision: Accepted.

Reflection: The line-ending problem is something I wouldn't have noticed until the PR showed every line as changed. The double-click case was also new to me: a check in Python alone isn't enough when two requests arrive at the same time.

Responsible Person: Diego Chidiac

4. Task 6.1 and 6.2: Testing without a frontend and the risk to the shared database

Purpose of AI Use: Finding out how to test the backend before the UI existed, and whether testing against our shared Supabase database was safe.

Chat Link or Prompt/Response:

Prompt: can i test the new features out by running it locally ? Or i cant see bc the frontend hasnt been done? And if it creates job in the supabase is it so wrong, can't i juste delete it after ?

Response: Yes. You test the backend by calling its endpoints directly, without the browser UI. I ran this exact flow against a live server here and it works: apply → success, apply again → "already applied", then the list shows one application with status Applied.

The `/docs` page won't work for this. FastAPI's `/docs` page normally lets you try endpoints by hand. But its "Authorize" button sends the login as a form, and your team's `/api/login` expects JSON, so it fails with a 422. [...] To actually call the endpoints, use **try-us06.sh**.

Is it so bad if it creates `jobs` on Supabase? It isn't a disaster, and yes, you can clean it up. The risk is that it fails quietly. `create_all()` only creates tables that don't exist yet; it never fixes a table that exists with different columns. So:

- **If my `jobs` table lands on Supabase first:** when your teammate's job model runs later, his table is skipped because the name is already taken. His extra columns never get created, and his code crashes with "column does not exist" without him knowing why.
- **If his `jobs` table is already on Supabase:** your code is the one that could crash, if his table doesn't have my `is_open` or `category` columns.

The zero-risk alternative is a local Postgres, so Supabase is never touched. That's what the tests and CI use.

AI-Suggested Content:

- A test script that walks through the whole apply flow with curl.
- SQL to insert a test job by hand.
- Cleanup steps if the tables were created on Supabase.

Validation: The AI ran the script against a live server before sending it. Its output showed a 201 on the first application and "You have already applied to this job." on the second. It also confirmed the /docs limitation by sending a form login, which returned 422.

Decision: Accepted. I postponed manual testing and kept the US-06 branch off Supabase until the real job model was merged.

Reflection: My first instinct was to test on the shared database and delete things after. The AI's explanation of create_all() showed me the danger wasn't the leftover data but a wrong table silently breaking a teammate's code.

Responsible Person: Diego Chidiac

5. US-07: Tasks 7.1 and 7.3 (status updates and history), backend for 7.2

Purpose of AI Use: Letting recruiters update an application's status, recording every change with a timestamp, and providing the data the recruiter screen needs.

Chat Link or Prompt/Response:

Prompt: let's just move to us07 we'll test everything once i get job.py and the listings

Response: Moving to US-07. I'll do 7.1 and 7.3 together, because every status change has to write a history entry anyway. I'll also add the "applicants for my posting" endpoint that the 7.2 recruiter screen needs. [...] All 105 tests pass: the 84 from before plus 21 new ones.

7.1: changing the status (#29), `PATCH /api/applications/{id}/status` with `{ "status": "Interview" }`.

- **Only the recruiter who posted that job** can do it. The applicant, other recruiters and other job seekers all get 403.
- An unknown status like "Hired" is rejected (422) before any code runs. [...] Setting the status it already has gives 400.

7.3: status history (#31)

- A new table, `application_status_history`, with one row per change: old status, new status, who changed it, and when. Rows are never edited.
- **I changed the 6.2 apply endpoint:** submitting now also writes the first entry (none → Applied). It's saved in the same commit as the application, so an application can never exist without its history.

7.2: the backend the recruiter screen needs (#30): `GET /api/jobs/{job_id}/applications` lists all applicants for one posting.

AI-Suggested Content:

- PATCH /api/applications/{id}/status, restricted to the job's recruiter.
- An application_status_history table, written in the same transaction as each change.
- GET /api/applications/{id}/history.
- GET /api/jobs/{job_id}/applications for the 7.2 screen.
- 21 tests.

Validation: 105 tests passed. I questioned whether 7.2 was really done, since the first summary listed it next to the completed tasks. The AI confirmed that only its backend was done.

Decision: Accepted. The PR closes #29 and #31 only; Task 7.2 stays open for the frontend.

Reflection: The code was correct, but the way it was summarized could have led me to close a task that wasn't done. Checking the AI's claims against the actual issue titles is part of the validation too.

Responsible Person: Diego Chidiac

6. US-04: Task 4.1 (job posting schema) and Task 4.2 (job posting API)

Purpose of AI Use: Building the official jobs table and the recruiter endpoints to manage postings, after being reassigned to US-04.

Chat Link or Prompt/Response:

Prompt: I'm now assigned to do us04, well the backend (4.1 and 4.2), we'll need a new branch and I setup the model job.py so we won't need to wait for us06 and us07 [attached screenshot of US-04 and its tasks]

Response: Switching to US-04. Your branch needs to start from `main`, without the US-06/07 work, so I'll rebuild from the original code your teammates pushed. [...] All 92 tests pass: the 56 originals plus 36 new ones. Before handing it over, I'll check that your US-06/07 code still works on top of this new job model, simulating the future merge: [...] With everything combined, all 141 tests pass, so US-06/07 will work with this job model after the merge.

Endpoint	Who	Does
POST /api/jobs	recruiters	create a posting (starts open)
GET /api/jobs/mine	recruiters	their own postings, open and closed
GET /api/jobs/{id}	any logged-in user	one posting; closed ones only for the owner
PUT /api/jobs/{id}	owner	edit
POST /api/jobs/{id}/close, /reopen	owner	stop or resume applications
DELETE /api/jobs/{id}	owner	delete permanently

Every "change" endpoint goes through one shared ownership check, which returns 403 for job seekers and for other recruiters.

Route order matters: FastAPI checks routes top to bottom. If `/jobs/{job_id}` came first, a request to `/jobs/mine` would match it, try to read "mine" as a number, and fail with a 422.

`extra="forbid"` rejects unknown fields. The important case is `recruiter_id` or `is_open` in the body: without this, a client could try to create a job owned by someone else.

Right now, deleting a job also deletes its applications (and their history). Once US-06 is merged, I'd suggest blocking delete when a posting has applicants ("close it instead"), so recruiters can't wipe job seekers' application records by accident.

[attached ai coded-files: models/job.py, schemas/job.py, routers/job.py, tests/test_job_model.py, tests/test_jobs.py, app.py, models/__init__.py, schemas/__init__.py]

AI-Suggested Content:

- A Job model with job-type and salary-range constraints.
- Seven job posting endpoints with a single ownership check.
- 36 tests.
- A simulated merge with US-06/US-07 to check compatibility.
- A future rule to block deleting jobs that have applicants.

Validation: 92 tests passed on the branch, and 141 in the simulated merge with US-06 and US-07. CI passed on the PR. I started the backend against Supabase and created a real job with POST /api/jobs, which returned "id": 1 with every field as sent.

Decision: Accepted. The rule to block deleting jobs with applicants was left for later.

Reflection: The compatibility check before opening the PR was valuable: it proved that replacing the temporary job model wouldn't break the work I had already done. I kept the field names US-06 relied on so the merge would be simple.

Responsible Person: Diego Chidiac

7. Cleaning up an incompatible table on Supabase

Purpose of AI Use: Deciding what to do with an existing applications table found on the shared database before creating the jobs table.

Chat Link or Prompt/Response:

Prompt: I already have an application table and it's empty

Then: here's what i have [screenshot of the table's columns: id, user_id, job_id, resume_id, submitted_at, status]

Response: That table isn't the one defined in your code. It was created by someone else, probably a teammate's early version or something from Sprint 1. It differs from what the US-06 code expects:

Existing table	Mine (US-06)
user_id	applicant_id
submitted_at	applied_at
—	updated_at
no rules visible	unique (job, applicant) + allowed-status check

If it stays, US-06 will crash on Supabase with errors like *"column applicant_id does not exist"*, because `create_all()` never fixes an existing table.

Since it's empty, drop it: In the SQL Editor: `DROP TABLE applications;` If it fails with *"other objects depend on it"*, send me the error rather than adding `CASCADE`.

I still need the first query's result, the list of all tables. If that table's `job_id` points at a `jobs` table, then `jobs` may already exist too. In that case, starting the US-04 backend would skip creating it and you'd keep the wrong version.

AI-Suggested Content:

- SQL queries to list the tables and inspect a table's columns.
- A comparison of the existing table with the US-06 model.
- Dropping the table after informing the team.

Validation: Inspected the table myself with the SQL queries provided and confirmed it was empty before dropping it.

Decision: Accepted.

Reflection: Because the table was empty, dropping it cost nothing, but without the column comparison I would have kept it and US-06 would have failed later with a confusing error. Checking with the team first mattered because the database is shared.

Responsible Person: Diego Chidiac

8. Resolving merge conflicts after a squash merge

Purpose of AI Use: Fixing the conflicts on the US-07 pull request after US-06 was merged into main.

Chat Link or Prompt/Response:

Prompt: okay i merged the pr for us06 now the pr for us07 says we have conflicts, idk if it's bc i see the commits of the us06 in the pr of us07

Response: You're right that the two are connected. [...] If the PR was merged with **"Squash and merge"** (or "Rebase and merge"), GitHub didn't put your original US-06 commits into `main`. It created **one new commit** with the same changes. So `main` now has a *copy* of US-06, while your US-07 branch has the *originals* plus your US-07 edits on top.

AI-Suggested Content:

- An explanation of why squash merging causes these conflicts.
- A check that no reviewer edits would be lost before resolving.
- Commands to keep the US-07 version of each conflicted file.
- A command to search for leftover conflict markers.

Validation: Before I resolved anything, the AI simulated both merges in a test repository and all tests passed (120 and 141). I caught the duplicate __all__ line myself, and confirmed the duplicated import with git diff --cached. The AI's first conflict-marker search also matched files inside venv/, so it replaced it with git grep on tracked files only.

Decision: Modified and accepted.

Reflection: Merge conflicts were the part where my own review mattered most: the AI's instructions were right, but I still made a mistake applying them, and I only caught it because I read the file before committing. The squash-merge explanation will help the team avoid this on the next PRs.

Responsible Person: Diego Chidiac